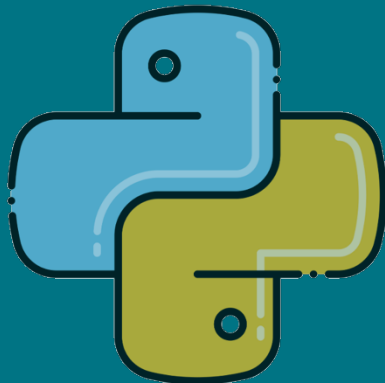

AGENDA 3

Introdução a linguagem de Programação PYTHON



SEAD- Superintendência de Ensino e Pesquisa nas
modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO
EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO DE SISTEMAS I

Expediente

Autores:

PAULO EDUARDO CARDOSO ANDRADE

TATIANE TOLENTINO DE ASSIS

Revisão Técnica:

Kelly Cristiane de Oliveira Dal Pozzo

São Paulo – SP, 2025



MOMENTO REFLEXÃO



A cada manhã, Joana acorda cedo para organizar sua rotina antes de sair para a escola. Ela gosta de manter tudo em ordem: anotar as tarefas do dia, controlar os gastos de sua mesada, calcular quanto tempo tem para cada atividade. Ao conversar com seus amigos, percebe que muitos deles também enfrentam dilemas parecidos: como poupar dinheiro para comprar algo que desejam, como planejar o tempo para estudar e brincar, ou até como decifrar o resultado de uma prova em que as questões parecem complexas. Mas, apesar de todos recorrerem a listas, calculadoras ou aplicativos para ajudar, poucos imaginam que, por trás dessas soluções, existe algo

fundamental: um pensamento lógico e organizado, que transforma problemas cotidianos em ações simples.

Certa vez, Joana se deparou com um desafio interessante. Seu irmão mais novo queria saber quantas moedas ele tinha no cofre, mas estava confuso sobre como calcular. Joana decidiu ajudá-lo, criando uma sequência de passos: primeiro, separou as moedas por valores, depois somou cada grupo e, por último, contou tudo junto. Enquanto explicava, percebeu que estava construindo um roteiro, detalhando cada etapa para tornar a tarefa compreensível. O irmão, fascinado, aprendeu a montar o cálculo e começou a pensar em outras formas de organizar suas tarefas: como calcular o tempo de brincadeira ou comparar os resultados das provas de matemática com facilidade.

Joana então refletiu:

"É como se eu estivesse contando uma história para o computador, mostrando como ele deve agir em cada fase, escolhendo o que guardar, somar ou transformar."

Assim, percebeu que muitas soluções tecnológicas partem desse mesmo princípio - um roteiro, com instruções claras, feito por pessoas, que depois é traduzido em códigos que o computador pode entender.

Quantas vezes, em seu dia a dia, você já tentou organizar uma tarefa, calcular um resultado ou transformar um problema em etapas simples?

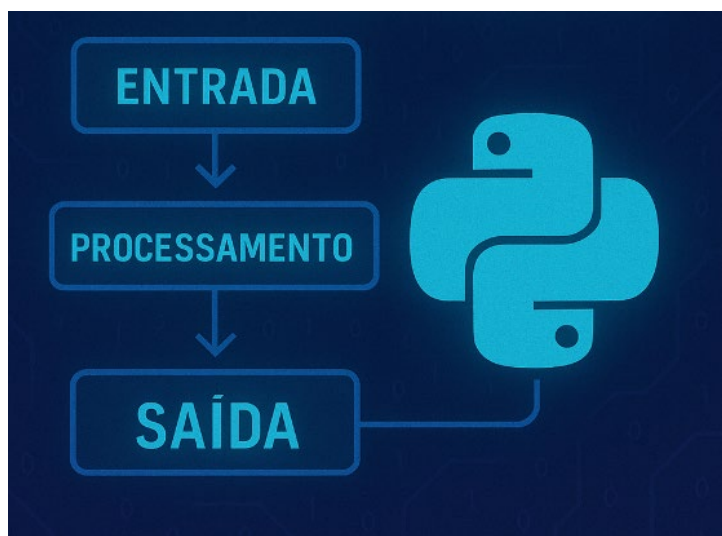
E se você tivesse que criar um roteiro para o computador resolver uma dessas tarefas, como você começaria?

O que facilitaria esse processo? Pense nisso.



PORQUE APRENDER?

Quando nos deparamos com a matemática na escola, aprendemos desde cedo a lidar com números, reconhecer quantidades e realizar operações fundamentais, como somar e dividir. Aos poucos, passamos a usar letras para representar valores, criando fórmulas que facilitam a resolução de problemas e ampliam nossas possibilidades de raciocínio. A programação, de maneira semelhante, nos oferece ferramentas para organizar e solucionar desafios, trazendo à tona a importância das variáveis, operadores e do fluxo lógico dos programas.



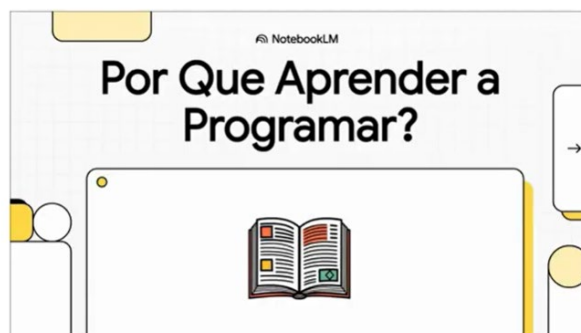
No universo do desenvolvimento de sistemas, especialmente utilizando Python, esses fundamentos são essenciais para criar aplicações que refletem situações do cotidiano e do mundo profissional. Imagine um ambiente de trabalho onde é preciso automatizar cálculos, processar pedidos, controlar estoques ou gerar relatórios precisos. Tudo isso parte de uma **estrutura básica: receber dados, processá-los de forma inteligente e apresentar resultados claros e confiáveis**. Ao aprender a organizar informações em **variáveis**, aplicar **operadores** para transformar esses dados e desenhar a trajetória de **entrada, processamento e**

saída, o aluno se prepara para elaborar algoritmos que tornam o fluxo de trabalho mais eficiente.

Além do conhecimento técnico, compreender esses fundamentos permite criar soluções personalizadas para muitos problemas práticos. Por exemplo, ao desenvolver um programa simples que calcula a média de notas de alunos, o profissional utiliza variáveis para guardar os valores, operadores para realizar o cálculo e comandos para mostrar o resultado de forma organizada. Essa experiência vai além do exercício escolar, aproximando o aluno do perfil de terminalidade esperado, onde é necessário converter necessidades reais em procedimentos programáveis, utilizáveis em Ambiente de Desenvolvimento Integrado (IDE), com técnicas de versionamento e organização lógica.

Dominar conceitos como variáveis, operadores e o esquema de entrada-processamento-saída representa o início de uma jornada que facilita a conexão entre teoria e prática. Gradualmente, o aluno compreende que programar é como aprender a falar um novo idioma: começa com pequenos passos, repetindo estruturas simples, até ganhar confiança para explorar projetos maiores, automatizar tarefas, analisar dados ou propor soluções inovadoras. O Python, por sua simplicidade e poder, torna esse processo mais direto e moderno, proporcionando ao estudante o ambiente ideal para experimentar, errar, corrigir e evoluir.

Assista o vídeo que traz uma introdução sobre o assunto:



Disponível em: <https://www.youtube.com/watch?v=mUE33sCH1Uk>

Acessado em: 27/02/2026.



PARA COMEÇAR O ASSUNTO

Joana, sempre organizada, decidiu que era hora de ajudar sua família a planejar melhor o orçamento do mês. Depois de ensinar ao irmão como calcular as moedas do cofrinho, ela percebeu que poderia criar uma solução ainda mais prática usando tecnologia: um pequeno programa capaz de registrar as receitas e despesas de cada membro da casa, calcular automaticamente o saldo e mostrar, em tempo real, se haveria dinheiro suficiente para a compra de um novo eletrodoméstico que todos desejavam.

No início, Joana reuniu todos ao redor da mesa, cada um trazendo anotações de gastos e valores recebidos. Enquanto discutiam, ela fez um esboço simples:

“Primeiro, precisamos saber quanto cada um recebeu — isso será a entrada do programa. Depois, vamos listar todas as despesas — esse será o processamento, onde o computador soma e subtrai os valores. Por fim, precisamos que o resultado seja apresentado de forma clara, mostrando quanto restou para as futuras compras.”

Ao explicar, Joana sentiu que estava montando um roteiro, transformando tarefas em passos lógicos, tal como havia feito manualmente com seu irmão.

Ela abriu o Python em seu notebook, criou variáveis para guardar cada valor, utilizou operadores aritméticos para calcular os resultados e, ao final, exibiu na tela o saldo atualizado. A família ficou encantada: ver todas as informações organizadas, saber exatamente o que podia ser comprado, e ainda poder planejar próximos gastos, tornou-se não apenas um exercício útil, mas também divertido. Joana percebeu que entender o ciclo de entrada, processamento e saída era mais do que uma técnica de programação; era transformar um problema real em uma solução concreta e eficiente.

Esse mesmo método serve para situações do cotidiano e para desafios no mundo do trabalho. De controlar o estoque de mercadorias a calcular o tempo necessário para concluir tarefas em uma fábrica, tudo começa com a organização das informações e a aplicação de lógica, utilizando os fundamentos da programação. Ao longo desta agenda, você aprenderá, com a experiência de Joana, como utilizar variáveis e operadores para criar programas que facilitam tarefas, automatizam cálculos e tornam decisões mais precisas.

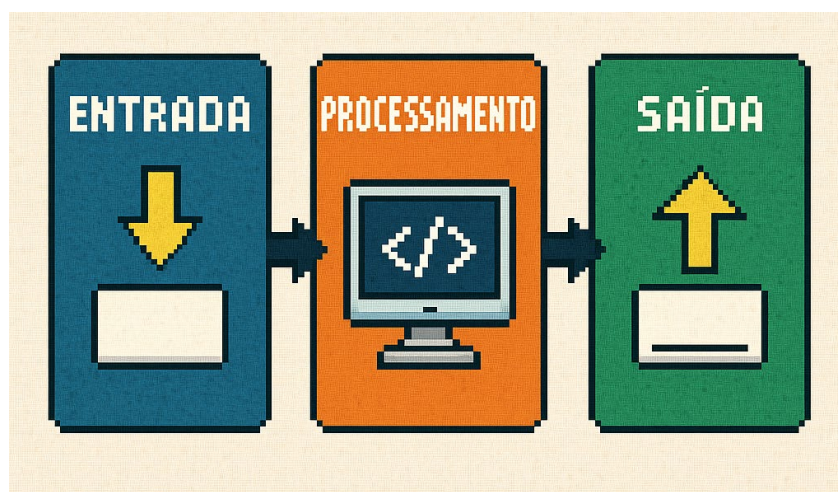
E no seu dia a dia: que tarefa você gostaria de automatizar? Como o uso de variáveis, operadores e um roteiro lógico pode tornar sua vida ou seu trabalho mais fácil, organizado e eficiente?



MERGULHANDO NO TEMA

Entrada, Processamento e Saída

Em qualquer paradigma de programação, seja ele procedural, orientado a objetos ou funcional, existe um ciclo central que une todos os sistemas desenvolvidos: a estrutura de entrada, processamento e saída. Essa sequência consiste na espinha dorsal dos algoritmos, conectando teorias de lógica de programação à prática do desenvolvimento de soluções.



A fase de entrada é o ponto de partida para todos os programas. Imagine um cenário em que um sistema precisa captar informações de um usuário, como o valor das horas trabalhadas em uma empresa ou as vendas realizadas em uma loja virtual. Essas informações podem ser inseridas manualmente pelo teclado, extraídas de arquivos como planilhas ou CSV, captadas por sensores embarcados em dispositivos IoT, ou até mesmo recebidas por meio de integrações externas, como APIs ou bancos de dados. O que define a etapa de entrada é o recebimento de dados brutos, preparados para serem tratados pelo sistema.

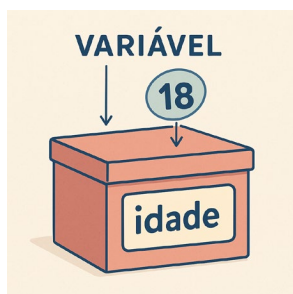
Após a obtenção dos dados, inicia-se o processamento, considerado o coração lógico de todo programa. Nessa etapa, o sistema deve aplicar regras de negócio, realizar cálculos aritméticos, transformar e combinar valores, definir limites e condições específicas, além de organizar resultados intermediários para análise futura. Seja o cálculo do desconto progressivo em uma compra, a atualização do saldo de um cliente ou a filtragem de registros para um relatório, o processamento emprega os operadores e as estruturas de decisão, colocando em prática o conhecimento algorítmico do programador.

Por fim, alcançamos a etapa da saída, que representa a materialização das informações trabalhadas. É o momento em que o usuário ou o sistema externo recebe uma resposta plausível, detalhada e adequada ao contexto solicitado. A saída pode ser apresentada de diversas formas: um valor exibido na tela indicando o “Total a pagar”, um relatório formatado com casas decimais e unidades de medida, um arquivo pronto para impressão, a transmissão de resultado a um outro software ou até mesmo o envio de alertas por dispositivos conectados.

Para que esse ciclo funcione, dependemos das variáveis, estruturas fundamentais para guardar e organizar informações durante o programa. Elas funcionam como caixas nomeadas, armazenando valores de entrada, resultados intermediários e produtos finais. Assim, garantimos que as etapas estejam conectadas e mantemos a organização dos dados ao longo de todo o processo de criação de soluções.

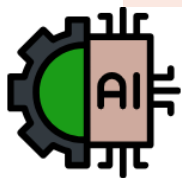
Agora que conhecemos o conceito norteador da lógica de programação, vamos explorar em detalhe o que são variáveis, como utilizá-las no Python e por que elas são indispensáveis para que o ciclo Entrada–Processamento–Saída funcione corretamente.

Variáveis — Guardando Informações no Programa



Quando colocamos em prática o ciclo de entrada, processamento e saída, percebemos que as variáveis são indispensáveis em cada etapa desse processo. Elas funcionam como caixas com etiquetas, que nós mesmos nomeamos para armazenar valores que precisam ser utilizados ao longo do programa. Essa prática facilita muito a organização das informações, pois sempre que precisamos acessar um dado, basta chamarmos a variável pelo nome que definimos.

Ao programarmos em Python, a utilização de variáveis torna-se ainda mais simples e intuitiva. Diferente de outras linguagens, não precisamos nos preocupar inicialmente com o tipo de dado — como inteiro ou texto —, pois o próprio Python identifica automaticamente o tipo do valor que está sendo armazenado. Podemos, por exemplo, atribuir diretamente o número de alunos de uma turma com a instrução “quantidade_alunos = 32”, ou informar o nome de um participante usando “nome = 'Ana'”. Essas operações são rápidas, eliminando barreiras de entrada para quem está começando ou para projetos que exigem agilidade.



Utilize ferramentas de Inteligência Artificial para pesquisar sobre tipos de variáveis. Pesquise os tipos de variáveis mais comuns para desenvolvimento de um sistema.

Dar nomes significativos às variáveis é também uma forma de comunicação eficiente entre nós, programadores, e com colegas que possam revisar ou dar manutenção aos nossos códigos no futuro. Ao optarmos por nomes como “idade”, “salario_mensal” ou “preco_total_compra”, estamos dizendo exatamente o que aquele valor representa, evitando confusões e minimizando erros. Essa atenção aos nomes faz parte das boas práticas do Python, recomendadas pela [PEP 8](#), que orienta o uso de `snake_case` para facilitar a leitura e manter a consistência. Além disso, ao especificar detalhes como unidades no nome — “temperatura_c” para graus Celsius ou “distancia_km” para quilômetros — conseguimos tornar o programa ainda mais completo e autoexplicativo.

**Snake_case**

É a convenção de nomenclatura na qual cada espaço é substituído por um caractere de sublinhado (`_`) e as palavras são escritas em letras minúsculas.

Durante toda a elaboração de um programa, as variáveis desempenham papéis essenciais. Na entrada, elas recebem e armazenam os dados que digitamos ou importamos, como informações de clientes ou parâmetros para cálculos. No processamento, guardam resultados intermediários, como soma parcial de vendas ou o desconto calculado em uma transação. E, por fim, na saída, são as responsáveis por apresentar valores claros e organizados para o usuário, seja exibindo mensagens na tela, gerando relatórios ou exportando para arquivos.

Regras de nomeação (Python)

Não pode iniciar com número: Comece com letra ou `_` underscore (`_`).



idade, `_temp` |



1aluno

Use apenas letras, números e underscore (`a-z`, `A-Z`, `0-9`, `_`): sem espaços.



quantidade_alunos



quantidade alunos

Case-sensitive: diferencia letras maiúsculas e minúsculas

Por exemplo: idade, Idade e IDADE são nomes diferentes.

Proibido o uso de Palavras Reservadas

Não use palavras reservadas do Python, por exemplo: if, for, class, def, while...

Acentos e cedilha

O uso de acentuação nas palavras tecnicamente é permitido, mas **evite** por portabilidade: prefira salario a salário.

Boas práticas (estilo Python/PEP 8)

- **snake_case** para variáveis: Exemplos: `nota_final`, `preco_unitario`.
- **Seja específico**: `preco_total_compra` é melhor que `preco`.
- **Unidade no nome quando fizer sentido**: Exemplos: `temperatura_c`, `distancia_km`, `taxa_percentual`.
- **Consistência** no projeto: não misture `precoTotal` com `preco_total`.
- **Evite nomes genéricos**: evite (`tmp`, `dados`, `valor`) usar somente quando realmente temporários e de escopo curto.
- **Singular vs. plural**: `aluno` (um elemento), `alunos` (coleção).
- **Sem números mágicos no nome**: ao invés de (`taxa15`), prefira `taxa_desconto` e documente o valor.
- **Comentários curtos**: só quando o nome não bastar, priorize nomes que eliminem o comentário.

python

```
# Bons exemplos
idade = 18

salario_mensal = 3250.00

quantidade_alunos = 32

preco_total_compra = preco_unitario * quantidade

temperatura_c = 23.5
```

Esses cuidados deixam seu código legível, fácil de manter e menos propenso a erros, e preparam o terreno para construir programas melhores.

Durante toda a elaboração de um programa, as variáveis desempenham papéis essenciais. Na entrada, elas recebem e armazenam os dados que digitamos ou importamos, como informações de clientes ou parâmetros para cálculos. No processamento, guardam resultados intermediários, como soma parcial de vendas ou o desconto calculado em uma transação. E, por fim, na saída, são as responsáveis por apresentar valores claros e organizados para o usuário, seja exibindo mensagens na tela, gerando relatórios ou exportando para arquivos.



Variáveis são a memória de trabalho do seu programa. Sem elas, seria impossível manipular dados de forma eficiente e reutilizável.

Entrada de dados (input) e variáveis

Todo programa começa recebendo informações para trabalhar. Essas informações podem vir das mais variadas fontes: documentos, arquivos, bancos de dados, sensores, sistemas externos e até mesmo da internet. Porém, para os nossos estudos iniciais em Python, a forma mais simples e direta de capturar dados é utilizando a função `input()`, que realiza a entrada de dados pelo teclado.

Sempre que utilizamos `input()`, o valor digitado pelo usuário é interpretado como texto e armazenado como uma string (str). Em seguida, esse valor pode ser guardado em uma variável, que servirá de base para o processamento.

python

```
nome = input("Digite seu nome: ")
```

No exemplo apresentado, o usuário digitar será guardado dentro da variável `nome`, permitindo que seja usado em outros pontos do programa.

Python é dinamicamente tipado, mas o tipo importa!

Embora você não declare tipos explicitamente ao criar variáveis, cada valor tem um tipo, e isso define o que é possível fazer com ele (somar, concatenar, dividir, formatar, etc.). Trabalhar conscientemente com tipos evita erros como:

python

```
numero = int(input("Digite um número: "))
```

Portanto, cada valor possui um tipo, que define o que pode ser feito com ele, por exemplo, números permitem cálculos, textos permitem concatenação e formatação. Embora o Python não exija que você declare explicitamente o tipo de uma variável, é fundamental entender os tipos de dados mais usados, pois eles influenciam diretamente nas operações possíveis e evitam erros comuns.

Conversão de Valores (Casting)

Em Python, a função `input()` sempre retorna texto (`str`). Quando precisamos realizar cálculos ou comparações, é necessário converter esse texto para o tipo adequado. **Esse processo é chamado de casting.**

Veja um exemplo prático:

python

```
idade = int("18")      # resultado: 18
altura = float("1.75") # resultado: 1.75
```

No exemplo apresentado acima temos a declaração de duas variáveis, “idade” e “altura”, para realização de cálculos com essas variáveis é necessário realizar a conversão do padrão recebido pela linguagem, que é texto, para números. Neste caso a variável “idade” recebe um valor inteiro e a variável “altura” recebe um valor decimal, que na programação é conhecido como float.

Veja na tabela abaixo alguns dos principais tipos de dados do Python, exemplos de valores (literals), formas de conversão (casting) e seus usos mais frequentes para facilitar o entendimento:

Tipo	Exemplo de literal	Conversão (cast)	Uso comum
Str (texto- string)	"Ana", "42", "3.14"	<code>str(valor)</code>	Mensagens, entradas do usuário, formatação
Int (inteiro)	7, -3, 0	<code>int("42") = 42</code>	Contagens, idades, quantidades
float (real)	3.14, -0.5, 2.0	<code>float("3.14") = 3.14</code>	Médias, preços, medidas
bool (lógico)	True, False	<code>bool(valor)*</code>	Decisões (if), flags

Agora que já conhecemos um pouco mais sobre declaração de variáveis, vamos compreender um comando completo em Python. Veja a seguinte linha de código:

python

```
altura = float(input("Digite sua altura em metros (ex.: 1.75): "))
```

Vamos detalhar os comandos:

- **input():** utilizado para exibir a mensagem "Digite sua altura em metros (ex.: 1.75): ", em seguida faz a leitura do que o usuário digitar no teclado. Conforme já vimos anteriormente essa função, por padrão, retorna o valor como texto, mesmo que seja um número. O computador não consegue diferenciar isso sozinho, então é necessário passar essa informação através do casting.
- **float():** envolve o input() para converter o texto digitado em um número real. Assim, se o usuário digitar "1.75", o valor será convertido para 1.75 (tipo float). Em resumo realiza o casting.
- **Atribuição:** o resultado da conversão é guardado na variável altura. Assim, essa variável passa a armazenar o número informado pelo usuário, pronto para ser usado em cálculos. O operador igual é o responsável pela atribuição do valor na variável "altura".



o input() lê um texto, o float() transforma esse texto em número decimal, e a variável altura guarda esse valor para uso posterior.

Processamento em Python e Operadores Aritméticos

Depois de receber os dados (entrada), chega o momento de processá-los. O processamento é o coração do programa: é onde aplicamos regras, fazemos cálculos e transformamos dados brutos em informações úteis. É nessa etapa que os valores armazenados em variáveis são combinados, manipulados e organizados para gerar novos resultados.

Operadores Aritméticos

Os **operadores** funcionam como as operações matemáticas que já usamos no dia a dia: somar preços no mercado, calcular a média das notas de uma prova, dividir uma conta entre amigos ou até descobrir se um número é par ou ímpar. Em programação, cada uma dessas situações pode ser resolvida utilizando um operador adequado. Com eles, podemos:

- Somar e subtrair valores, como em uma calculadora.
- Multiplicar e dividir, seja para cálculos diretos ou para ajustar quantidades.
- Obter a parte inteira de uma divisão, ignorando as casas decimais.
- Descobrir o resto de uma divisão, útil em testes de paridade (número par ou ímpar).
- Elevar um número a uma potência, como calcular áreas, volumes ou crescimento exponencial.

Os operadores são a base do processamento em Python, e combinados com variáveis permitem transformar dados brutos em informações significativas.

No Python temos os principais operadores aritméticos:

Operador	Símbolo	Exemplo	Aplicações comuns
Adição	+	5 + 3	Somar valores, calcular totais
Subtração	-	10 - 4	Diferença entre números, troco, saldo
Multiplicação	*	7 * 6	Preço × quantidade, áreas, escalas
Divisão (resultado real)	/	8 / 2	Médias, divisão de contas, proporções
Divisão inteira	//	8 // 3	Partições inteiras, cálculo de lotes
Módulo (resto da divisão)	%	8 % 3	Testar paridade (par/ímpar), ciclos
Exponenciação (potência)	**	2 ** 3	Potências, áreas, juros compostos



Dicas Importantes:

- Use // quando quiser apenas a parte inteira da divisão.
- Use % quando precisar verificar se um número é divisível por outro.
- Use ** para cálculos de potência ou crescimento exponencial.

Vamos visualizar na prática um exemplo de uso de operadores.

- 1 – Abra o VSCode.
- 2 – Crie um novo arquivo de texto (New Text File).
- 3 – Utilize o recurso Salvar Como (Save As) para salvar o arquivo no formato Python no seu computador. Salve com o nome e extensão: Operadores.py.
- 4 – Digite o seguinte código:

python

```
nota1 = float(input("Digite a 1ª nota: "))
nota2 = float(input("Digite a 2ª nota: "))
media = (nota1 + nota2) / 2
```

No código apresentado acima temos a declaração e entrada de dados em duas variáveis, 'nota1' e 'nota2', ambas recebem um valor do tipo float que será digitado pelo usuário. Na terceira linha temos o primeiro exemplo de uso de operadores matemáticos, onde a variável média recebe a soma dos valores armazenados nas variáveis nota1 e nota2, em seguida divide o resultado da soma por 2.

Execute o código e veja o resultado no console:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Microsoft Windows [versão 10.0.19045.6466]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\kelly>C:/Users/kelly/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/kelly/OneDrive - Centro Paula Souza/Documentos/Operadores.py"
Digite a 1ª nota: 10
Digite a 2ª nota: 5

C:\Users\kelly>

```



Mas e o resultado? Não foi exibido???
Calma!!
Agora vamos para a saída!

Saída de Dados em Python

Depois de realizar o processamento, o programa precisa mostrar os resultados de alguma forma. Essa etapa é chamada de **saída**. A saída pode acontecer de diferentes maneiras: exibindo informações na tela, enviando dados para outro programa, registrando em um arquivo ou até mesmo armazenando em um banco de dados para uso futuro. Para os nossos estudos em Python, utilizaremos a função `print()`, que mostra as informações no terminal.

Continuando nosso exemplo anterior, vamos completar o código adicionando a linha correspondente ao comando de saída. A linha de comando é: `print("A média entre", nota1, "e", nota2, "é", media)`

Veja o código completo:

python

```

nota1 = float(input("Digite a 1ª nota: "))
nota2 = float(input("Digite a 2ª nota: "))
media = (nota1 + nota2) / 2
print("A média entre", nota1, "e", nota2, "é", media)

```

No exemplo acima o comando `print` será utilizado para exibir a informação no terminal, observe que os textos estão entre aspas, já as variáveis não. Para separação de cada item utilizamos vírgula.

Execute o código e verifique o resultado no terminal:

```

C:\Users\kelly>C:/Users/kelly/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/kelly/OneDrive - Centro Paula Souza/Documentos/Operadores.py"
Digite a 1ª nota: 5
Digite a 2ª nota: 6
A média entre 5.0 e 6.0 é 5.5

```

Não esqueça as duas primeiras informações devem ser digitadas.

Assim, a f-string garante que os valores apareçam no formato desejado, deixando o resultado claro e padronizado.

Com isso conhecemos como aplicar o conceito de entrada – processamento e saída na linguagem Python. Agora vamos conhecer alguns outros detalhes importantes para desenvolvimento com a linguagem.

Indentação em Python

Indentação é o recuo aplicado no início de uma linha de código por meio de espaços ou tabulações, utilizado para organizar visualmente as instruções de um programa e indicar quais comandos pertencem a um mesmo bloco lógico.

Na programação, a indentação é importante porque **melhora a legibilidade** do código, **facilita sua compreensão** por outras pessoas desenvolvedoras e contribui para a manutenção do software ao longo do tempo. Um código bem indentado permite identificar rapidamente estruturas como decisões, repetições e funções, tornando a lógica mais clara e estruturada.

No caso da linguagem Python, a indentação vai além da organização visual: ela faz parte da própria sintaxe da linguagem. Isso significa que o Python utiliza os espaços no início das linhas para definir blocos de código. Diferente de outras linguagens que empregam chaves { } ou palavras-chave específicas para delimitar blocos, no Python é a quantidade de espaços que determina quais instruções pertencem a uma mesma estrutura.

Veja um exemplo de erro comum:

python

```
# Entrada de dados
nota1 = float(input("Digite a 1ª nota: "))
nota2 = float(input("Digite a 2ª nota: ")) # Vai causar erro de indentação
```



Qualquer desalinhamento entre linhas que deveriam estar no mesmo bloco fará com que o programa apresente erro, pois o interpretador Python entende que a estrutura do código está incorreta.

Mais à frente, na agenda de desvio condicionais (if/else), vamos explorar melhor como a indentação organiza os blocos de decisão. Por enquanto, já saiba: mantenha seu código sempre alinhado à esquerda, exceto quando precisar criar blocos internos.

Comentários

Em Python, um **comentário** é iniciado com o símbolo **#**. Tudo que estiver escrito à direita desse símbolo, na mesma linha, será ignorado pelo interpretador. Isso significa que o comentário não altera a execução do programa: ele serve apenas para documentar, explicar ou organizar o código.

Por que usar comentários?

- **Documentar o código:** explicando a lógica de um cálculo, o papel de uma variável ou a função de um trecho.
- **Facilitar a leitura:** deixando o código mais compreensível para você e para outras pessoas.
- **Desativar trechos temporariamente:** útil para testes sem precisar apagar código.

Veja como fica o nosso exemplo prático com o código comentado:

python

```
# Entrada de dados
nota1 = float(input("Digite a 1ª nota: "))
nota2 = float(input("Digite a 2ª nota: "))
# Processamento
media = (nota1 + nota2) / 2
# Saída usando f-string com formatação
print(f"A média entre {nota1:.1f} e {nota2:.1f} é {media:.2f}")
```

Comentários de múltiplas linhas (docstrings)

Além do **#**, também podemos usar três aspas duplas (`""" ... """`) ou simples (`''' ... '''`) para escrever comentários maiores, ocupando várias linhas. Tecnicamente, o Python trata isso como uma string não atribuída, mas é usado amplamente como comentário de bloco.

Veja um exemplo:

python

```
# Comentário de linha única
nota1 = float(input("Digite a 1ª nota: "))
"""
Comentário de múltiplas linhas
- Explicando o objetivo do programa
- Mostrando instruções passo a passo
"""
nota2 = float(input("Digite a 2ª nota: "))
```

No VSCode também é possível utilizar as teclas de atalho “Ctrl +” para comentar o código Python.

Uso do ponto (.) em números decimais

Em Python (assim como em muitas linguagens de programação), o **separador decimal é o ponto (.)**. Exemplo: 1.75, 3.5, 10.0, isso significa que, para representar números reais, devemos sempre usar o ponto. Se tentarmos digitar 1,75, o Python não entenderá como número, e sim como texto, gerando um erro.



VOCÊ NO COMANDO

Agora que já conhecemos os conceitos básicos, analise a seguinte situação problema:

Imagine que você foi convidado(a) a ajudar no desenvolvimento de um pequeno sistema para uma loja que está iniciando a informatização de suas vendas.

Atualmente, o atendente precisa calcular manualmente o valor total das compras realizadas pelos clientes, o que pode gerar erros e atrasar o atendimento. Para resolver essa situação, a loja decidiu criar um programa simples em Python que automatize esse processo.

Sua missão será desenvolver uma solução que permita registrar as informações de um produto vendido e calcular automaticamente o valor total da compra.

Para isso, o programa deverá solicitar ao usuário o nome do produto, o preço unitário e a quantidade adquirida. Em seguida, será necessário garantir que os dados informados estejam nos tipos corretos para que o

cálculo possa ser realizado. Por fim, o sistema deverá calcular o valor total da compra e apresentar o resultado de forma clara para o usuário.

Para solucionar o problema apresentado, vamos realizar uma programação em Python, para isso é necessário criar um arquivo com a extensão .py. Esse será o local onde vamos escrever nosso código.

1. Abra o VS Code.
2. No menu lateral, clique em Explorer (ícone de pastinha).
3. Clique em New File (ou use o atalho Ctrl + N para criar e depois Ctrl + S para salvar).
4. Escolha um nome para o arquivo, por exemplo: primeiro_programa.py.

Pronto! Agora você já tem um arquivo Python e pode começar a escrever seus comandos.

Veja se você conseguiu resolver os desafios:

Verifique o código completo contendo entrada, processamento e saída:

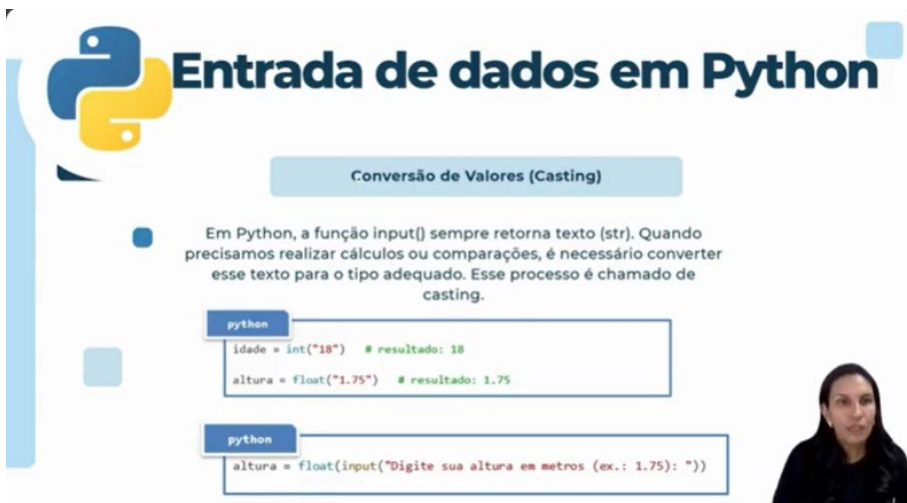
python

```
# Entrada das informações
produto = input("Digite o nome do produto: ")
preco = float(input("Digite o preço unitário (ex.: 4.50): "))
quantidade = int(input("Digite a quantidade comprada: "))

# Processamento (realização do cálculo)
total = preco * quantidade

# Saída - Exibindo as informações.
print(f"Você comprou {quantidade}x {produto}(s).")
print(f"Valor total da compra: R$ {total:.2f}")
```

Assista a vídeo aula da professora Tatiane Assis:



Entrada de dados em Python

Conversão de Valores (Casting)

Em Python, a função `input()` sempre retorna texto (`str`). Quando precisamos realizar cálculos ou comparações, é necessário converter esse texto para o tipo adequado. Esse processo é chamado de casting.

```
python
idade = int("18") # resultado: 18
altura = float("1.75") # resultado: 1.75
```

```
python
altura = float(input("Digite sua altura em metros (ex.: 1.75): "))
```

Disponível em: <https://www.youtube.com/watch?v=iFE9Zo8leak>

Acessado em 27/02/2026.